11. SIMD 및 하드웨어 가속 최적화 분석

개요

Stwo 프로젝트는 Circle STARK 증명 시스템의 성능을 극대화하기 위해 다양한 SIMD(Single Instruction, Multiple Data) 명령어와 하드웨어 가속 기법을 활용합니다. 이 문서는 Stwo에서 구현된 벡터화 기법, 플랫폼별 최적화 전략, 그리고 하드웨어 특성을 활용한 성능 향상 방법들을 상세히 분석합니다.

SIMD 백엔드 아키텍처

백엔드 구조

Stwo는 CPU와 SIMD 두 가지 백엔드를 제공하며, SIMD 백엔드는 벡터화된 연산을 통해 성능을 크게 향상시킵니다:

```
// SIMD 백엔드 구조
pub struct SimdBackend;

impl Backend for SimdBackend {}
impl BackendForChannel<Blake2sMerkleChannel> for SimdBackend {}
impl BackendForChannel<Poseidon252MerkleChannel> for SimdBackend {}
```

벡터 레인 구성

SIMD 구현은 16개의 레인을 가진 벡터를 기본 단위로 사용합니다:

```
pub const LOG_N_LANES: u32 = 4;

pub const N_LANES: usize = 1 << LOG_N_LANES; // 16 레인
```

필드 연산의 벡터화

M31 필드 벡터화

M31 기본 필드의 SIMD 구현은 16개의 M31 원소를 동시에 처리합니다:

```
#[repr(transparent)]

pub struct PackedM31(Simd<u32, N_LANES>);


impl PackedM31 {

pub const fn broadcast(M31(value): M31) -> Self {

Self(Simd::splat(value))

}

pub fn from_array(values: [M31; N_LANES]) -> PackedM31 {

Self(Simd::from_array(values.map(|M31(v)| v)))

}

}
```

플랫폼별 곱셈 최적화

Stwo는 다양한 하드웨어 플랫폼에 대해 특화된 곱셈 구현을 제공합니다:

AVX512 최적화 (x86_64)

```rust
#[cfg(all(target_arch = "x86_64", target_feature = "avx512f"))]
pub(crate) fn mul_avx512(a: PackedM31, b: PackedM31) -> PackedM31 {
mul_doubled_avx512(a, b.0 + b.0)
}

pub(crate) fn mul_doubled_avx512(a: PackedM31, b_double: u32x16) -> PackedM31 {
let a: __m512i = unsafe { transmute(a) };
let b_double: __m512i = unsafe { transmute(b_double) };
// 64비트 워드의 하위 절반에 짝수 32비트 워드 설정
let a_e = a;
let a_o = unsafe { _mm512_srli_epi64(a, 32) };
// 곱셈 수행
let prod_dbl_e: u32x16 = unsafe { transmute(_mm512_mul_epu32(a_e, b_dbl_e)) };
let prod_dbl_o: u32x16 = unsafe { transmute(_mm512_mul_epu32(a_o, b_dbl_o)) };
// 결과 조합
PackedM31(prod_lo) + PackedM31(prod_hi)
}
```

AVX2 최적화 (x86_64)

```rust
#[cfg(all(target_arch = "x86_64", target_feature = "avx2"))]
```

```
pub(crate) fn mul_avx2(a: PackedM31, b: PackedM31) -> PackedM31 {
// 256비트 벡터를 사용한 최적화된 곱셈
let [a0, a1]: [__m256i; 2] = unsafe { transmute(a) };
let [b0_dbl, b1_dbl]: [__m256i; 2] = unsafe { transmute(b_double) };
// 병렬 곱셈 수행
let prod0_dbl_e = unsafe { _mm256_mul_epu32(a0_e, b0_dbl_e) };
let prod1_dbl_e = unsafe { _mm256_mul_epu32(a1_e, b1_dbl_e) };
// 결과 인터리빙 및 조합
PackedM31(prod_lows) + PackedM31(prod_highs)
}
```

NEON 최적화 (ARM64)

```
#[cfg(all(target_arch = "aarch64", target_feature = "neon"))]
pub(crate) fn mul_neon(a: PackedM31, b: PackedM31) -> PackedM31 {
// NEON SIMD 명령어를 사용한 최적화
let c0: u32x4 = unsafe { transmute(vqdmull_s32(a0, b0)) };
let c1: u32x4 = unsafe { transmute(vqdmull_s32(a1, b1)) };
// 디인터리빙 및 결과 조합
let (mut c0_c1_lo, c0_c1_hi) = c0.deinterleave(c1);
c0_c1_lo >>= 1;
PackedM31(lo) + PackedM31(hi)
}
```

WebAssembly SIMD128 최적화

```
#[cfg(all(target_arch = "wasm32", target_feature = "simd128"))]
pub(crate) fn mul_wasm(a: PackedM31, b: PackedM31) -> PackedM31 {
// WebAssembly SIMD 명령어 활용
let c0_lo: u32x4 = unsafe { transmute(i64x2_extmul_low_u32x4(a0, b_double0)) };
let c0_hi: u32x4 = unsafe { transmute(i64x2_extmul_high_u32x4(a0, b_double0)) }
// 결과 처리 및 조합
PackedM31(even) + PackedM31(odd)
}
```

QM31 확장 필드 벡터화

QM31 확장 필드는 두 개의 CM31 벡터로 구성되어 복소수 연산을 벡터화합니다:

```
pub struct PackedQM31(pub [PackedCM31; 2]);

impl Mul for PackedQM31 {
type Output = Self;
fn mul(self, rhs: Self) -> Self::Output {
// Karatsuba 알고리즘을 사용한 벡터화된 곱셈
let ac = self.a() * rhs.a();
let bd = self.b() * rhs.b();
let bd_times_1_plus_i = PackedCM31([bd.a() - bd.b(), bd.a() + bd.b()]);
```

```
let ac_p_bd = ac + bd;
let ad_p_bc = (self.a() + self.b()) * (rhs.a() + rhs.b()) - ac_p_bd;
Self([l, ad_p_bc])
}
}
```

FFT 벡터화 최적화

Circle FFT SIMD 구현

Circle FFT는 SIMD 명령어를 활용하여 병렬로 처리됩니다:

```
pub fn compute_first_twiddles(twiddle1_dbl: u32x8) -> (u32x16, u32x16) {
// 첫 번째 레이어 트위들 계산
let t1 = simd_swizzle!(
twiddle1_dbl,
twiddle1_dbl,
[0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7]
);
// 레이어 0 트위들 계산
const NEGATION_MASK: u32x16 = u32x16::from_array([
0, P2, P2, 0, 0, P2, P2, 0, 0, P2, P2, 0, 0, P2, P2, 0
]);
let t0 = simd_swizzle!(t1, [...]) ^ NEGATION_MASK;
```

```
(t0, t1)
}
```

벡터 전치 최적화

FFT에서 사용되는 벡터 전치 연산은 병렬 처리로 최적화됩니다:

```
pub unsafe fn transpose_vecs(values: *mut u32, log_n_vecs: usize) {
let half = log_n_vecs / 2;
let values = UnsafeMut(values);
parallel_iter!(0..1 << half).for_each(|a| {
let values = values.get();
for b in 0..1 << (log_n_vecs & 1) {
for c in 0..1 << half {
let i = (a << (log_n_vecs - half)) | (b << half) | c;
let j = (c << (log_n_vecs - half)) | (b << half) | a;
if i >= j { continue; }
// 벡터 교환
let val0 = load(values.add(i << 4).cast_const());
let val1 = load(values.add(j << 4).cast_const());
store(values.add(i << 4), val1);
store(values.add(j << 4), val0);
}
}
```

```
});

}
```

배치 역원 계산 최적화

Montgomery의 트릭 벡터화

배치 역원 계산은 Montgomery의 트릭을 벡터화하여 구현됩니다:

```
pub fn batch_inverse_in_place<F: FieldExpOps>(column: &[F], dst: &mut [F]) {
const WIDTH: usize = 4;
let n = column.len();
if n <= WIDTH || !n.is_multiple_of(WIDTH) {
batch_inverse_classic(column, dst);
return;
}
// 첫 번째 패스: 인터리빙 방식으로 누적곱 계산
let mut cum_prod: [F; WIDTH] = array::from_fn(|_| F::one());
dst[..WIDTH].clone_from_slice(&cum_prod);
for i in 0..n {
cum_prod[i % WIDTH] *= column[i].clone();
dst[i] = cum_prod[i % WIDTH].clone();
}
// 역 누적곱 계산
```

```
let mut tail_inverses: [F; WIDTH] = array::from_fn(|_| F::one());
batch_inverse_classic(&dst[n - WIDTH..], &mut tail_inverses);
// 두 번째 패스: 최종 결과 계산
for i in (WIDTH..n).rev() {
dst[i] = dst[i - WIDTH].clone() * tail_inverses[i % WIDTH].clone();
tail_inverses[i % WIDTH] *= column[i].clone();
}
dst[0..WIDTH].clone_from_slice(&tail_inverses);
}
```

청크 기반 병렬 처리

대용량 데이터에 대해서는 청크 단위로 병렬 처리를 수행합니다:

```
pub fn batch_inverse_chunked<T: FieldExpOps + Send + Sync>(
column: &[T],
dst: &mut [T],
chunk_size: usize,
) {
#[cfg(not(feature = "parallel"))]
let iter = dst.chunks_mut(chunk_size).zip(column.chunks(chunk_size));
#[cfg(feature = "parallel")]
let iter = dst
.par_chunks_mut(chunk_size)
```

```
.zip(column.par_chunks(chunk_size));
iter.for_each(|(dst, column)| {
batch_inverse_in_place(column, dst);
});
}
```

비트 역순 최적화

SIMD 비트 역순 구현

비트 역순 연산은 16x16 블록 단위로 벡터화됩니다:

```
pub fn bit_reverse_m31(data: &mut [PackedBaseField]) {
let log_size = data.len().ilog2();
let a_bits = log_size - 2 * W_BITS - VEC_BITS;
let data = UnsafeMut(data);
parallel_iter!(0u32..(1 << a_bits)).for_each(|a| {
let data = unsafe { data.get() };
for w_l in 0u32..1 << W_BITS {
let w_l_rev = w_l.reverse_bits() >> (u32::BITS - W_BITS);
for w_h in 0..w_l_rev + 1 {
let idx = ((((w_h << a_bits) | a) << W_BITS) | w_l) as usize;
let idx_rev = bit_reverse_index(idx, log_size - VEC_BITS);
if idx > idx_rev { continue; }
```

```
// 16x16 블록 처리
let chunk0 = array::from_fn(|i| unsafe {
*data.get_unchecked_mut(idx + (i << (2 * W_BITS + a_bits)))
});
let values0 = bit_reverse16(chunk0);
// 결과 저장
for i in 0..16 {
unsafe {
*data.get_unchecked_mut(idx + (i << (2 * W_BITS + a_bits))) =
values0[i];
}
}
}
}
});
}
```

16원소 비트 역순

16개 원소의 비트 역순은 인터리빙 연산을 4번 반복하여 구현됩니다:

```
fn bit_reverse16(mut data: [PackedBaseField; 16]) -> [PackedBaseField; 16] {
// abcd:0123 => 0abc:123d 순열을 4번 적용
for _ in 0..4 {
```

```
let (d0, d8) = data[0].interleave(data[1]);

let (d1, d9) = data[2].interleave(data[3]);

// ... 나머지 인터리빙 연산

data = [d0, d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, d11, d12, d13, d14, d15];

}

data

}
```

프리픽스 합 최적화

Blelloch 알고리즘 벡터화

프리픽스 합 계산은 Blelloch의 병렬 알고리즘을 SIMD로 구현합니다:

```
pub fn inclusive_prefix_sum(

bit_rev_circle_domain_evals: Col<SimdBackend, BaseField>,

) -> Col<SimdBackend, BaseField> {

let mut res = bit_rev_circle_domain_evals;

let packed_len = res.data.len();

let (l_half, r_half) = res.data.split_at_mut(packed_len / 2);

// Up Sweep 단계

for ([l0, l1], [r0, r1]) in izip!(l_half.array_chunks_mut(), r_half.array_chunk

let (mut half_coset0_lo, half_coset1_hi_rev) = l0.deinterleave(*l1);

let half_coset1_hi = half_coset1_hi_rev.reverse();
```

```
up_sweep_val(&mut  half_coset0_lo, half_coset1_lo);

up_sweep_val(&mut  half_coset0_hi, half_coset1_hi);

}

// Down Sweep 단계

// ... 유사한 벡터화된 처리

res

}
```

하드웨어별 성능 특성

최적 청크 크기

각 필드 타입별로 경험적으로 결정된 최적 청크 크기를 사용합니다:

```
// Intel 155u 머신에서 경험적으로 결정된 최적 크기

pub(super) const PACKED_M31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 9; // 512

pub(super) const PACKED_CM31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 10; // 1024

pub(super) const PACKED_QM31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 11; // 2048
```

플랫폼별 기능 감지

컴파일 타임에 플랫폼별 기능을 감지하여 최적화된 구현을 선택합니다:

```
cfg_if::cfg_if! {

if #[cfg(all(target_arch = "aarch64", target_feature = "neon"))] {
```

```
mul_neon(self, rhs)
} else if #[cfg(all(target_arch = "wasm32", target_feature = "simd128"))] {
mul_wasm(self, rhs)
} else if #[cfg(all(target_arch = "x86_64", target_feature = "avx512f"))] {
mul_avx512(self, rhs)
} else if #[cfg(all(target_arch = "x86_64", target_feature = "avx2"))] {
mul_avx2(self, rhs)
} else {
mul_simd(self, rhs)
}
}
```

메모리 접근 최적화

정렬된 메모리 접근

SIMD 연산을 위해 메모리 정렬을 보장합니다:

```
impl PackedM31 {
/// # Safety
/// 포인터가 PackedM31과 동일한 정렬을 가져야 함
pub const unsafe fn load(mem_addr: *const u32) -> Self {
Self(ptr::read(mem_addr as *const u32x16))
}
```

```
/// # Safety
/// 포인터가 PackedM31과 동일한 정렬을 가져야 함
pub const unsafe fn store(self, dst: *mut u32) {
ptr::write(dst as *mut u32x16, self.0)
}
}
```

캐시 친화적 접근 패턴

데이터 접근 패턴을 캐시 라인에 맞게 최적화하여 메모리 대역폭을 효율적으로 활용합니다.

성능 향상 효과

벡터화 이득

- **16배 병렬성**: 16개 원소를 동시 처리
- **메모리 대역폭 활용**: 연속된 메모리 접근으로 캐시 효율성 향상
- **명령어 수준 병렬성**: 현대 CPU의 슈퍼스칼라 실행 활용

플랫폼별 최적화 효과

- **AVX512**: 512비트 벡터 연산으로 최대 성능
- **AVX2**: 256비트 벡터로 균형잡힌 성능
- **NEON**: ARM 플랫폼에서 효율적인 모바일 최적화
- **WebAssembly**: 브라우저 환경에서의 고성능 실행

결론

Stwo의 SIMD 및 하드웨어 가속 최적화는 Circle STARK 증명 시스템의 성능을 크게 향상시킵니다. 플랫폼별 특화된 구현, 벡터화된 필드 연산, 그리고 메모리 접근 최적화를 통해 현대 하드웨어의 성능을 최대한 활용합니다. 이러한 최적화 기법들은 대규모 증명 생성에서 실질적인 성능 향상을 제공하며, 다양한 하드웨어 플랫폼에서 일관된 고성능을 보장합니다.